

NASAQ AI - Project Status, Architecture & Handover

King Faisal University | Graduation Project Comprehensive Technical Summary

Project Name: Nasaq AI (نَسَاق)	Academic Baseline: M2 Milestone (Dr. Asrar)
Status Date: August - September 2026	Core Stack: React Flow, Laravel 11, FastAPI, Celery, Redis
Demonstrator Readiness: 74 / 100	Sharing Distribution: shr/ & shr.zip (0.44 MB ready)

1. What's in the Project & Distribution Package (shr/)

Nasaq AI is a full-stack, bilingual (Arabic RTL / English LTR) multi-agent workflow platform. Users visually construct agent pipelines on an interactive canvas, run them asynchronously, manage human approval checkpoints, and produce real durable outputs (Markdown, PDF, DOCX, and MP4 video).

Directory / Package	Tech Stack	Core Responsibilities & Capabilities
frontend/	React 18, React Flow, Vite, Tailwind CSS	Interactive DAG workflow editor, 7 specialized agent UI nodes, responsive bilingual shell (LTR/RTL), and full Admin Portal (/admin/overview, /admin/providers, /admin/system, /admin/users).
backend-laravel/	Laravel 11, Sanctum, PostgreSQL	Identity, authentication, role-based authorization (user/admin), workflow persistence, encrypted platform credentials, and authoritative JIT provider resolver.
ai-service-python/	FastAPI, Celery, Redis, FFmpeg	Asynchronous agent pipeline (Researcher, Writer, Video, Human Approval, Publisher, Email, Export), normalized OpenAI & Gemini adapters, bounded health probes.
infrastructure/	Docker Compose, Nginx	Six-service development and production topologies, Nginx reverse proxy configurations, and automated verification scripts.
Distribution (shr/)	Clean Package (0.44 MB)	Fully sanitized, isolated bundle containing all necessary code, start.bat, start.sh, bilingual README, and demo-ready .env (no leaked keys).

2. What's Added (Recent Implementations & Upgrades)

A. Phase 1 - Truthful Provider Control Plane (August 15, 2026):

- Authoritative Central Control: Moved LLM provider resolution from uncoordinated worker env vars into Laravel's database-backed admin configuration (AiProviderConfiguration and PlatformIntegration).
- Just-In-Time (JIT) Resolution: Celery worker fetches provider settings dynamically per run via GET /api/internal/runs/{run}/provider-resolution using a secure internal service token.
- Strict Zero-Leakage Credential Policy: Raw keys remain encrypted at rest; never passed in task payloads, URLs, browser APIs, or run logs. Passed strictly in worker process memory.
- Gemini Header Security: Migrated Gemini provider to standard x-goog-api-key headers, completely eliminating query parameter key leaks.
- Truthful Bounded Probes: Admin provider test buttons call a real timeout-limited FastAPI probe (POST /internal/providers/probe) rather than returning synthetic success.
- Celery Worker Healthcheck: Replaced inappropriate HTTP probe with a Celery/Redis inspect ping, resolving the container unhealthy state.

B. Multi-Agent Provider & Routing Redesign (August 25, 2026):

- Database-Driven Agent Routing: Added database catalog for 8 specialized agent categories with per-category encrypted configurations.
- Primary & Fallback Priorities: Support for multi-provider routing per agent with automatic fallback ordering.
- Admin Routing Controller: Added AiProviderRoutingController with password confirmation for activation and sanitized audit logging.

C. Clean Sharing Package Extraction (September 6, 2026):

- Isolated a 0.44 MB distribution package in shr/ and shr.zip (stripped all node_modules, vendor, .env, caches, and backups).
- Created pre-configured local .env with Demo Mode enabled by default (personal OpenAI/Tavily keys protected).
- Added 1-click launch scripts (start.bat for Windows, start.sh for Linux/Mac) and clear bilingual setup guides.

3. Where We Stopped (Current State & Active Blocker)

ACTIVE BLOCKER: Local PostgreSQL Volume Credential Mismatch on Rebuild
Following a Docker container rebuild, the persistent volume (postgres_data) retained the initial PostgreSQL password, causing Laravel to fail startup due to authentication rejection against the updated password in .env. Because the frontend container waits for Laravel health (depends_on: service_healthy), browser-based acceptance testing in English and Arabic is temporarily blocked on the developer machine until the volume or password is synchronized.

Summary of Stopped State:

- Automated Test Suite Status (100% Pass): Frontend: 39 tests pass; Python: 47 tests pass (1 host FFmpeg skipped); Laravel: 65 tests, 447 assertions pass; Compose configuration validation: pass.
- Container Health: Redis, Celery Worker, and FastAPI are running and healthy. Laravel and Frontend are pending database volume alignment.
- External Paid Calls: Paid OpenAI/Gemini runs were intentionally unverified to avoid costs; mock boundary contracts verified safe.
- Git Baseline: Files remain in workspace; initial repository commit baseline pending.

4. What's Remaining (Pending Requirements & Future Scope)

Area	Status	Description of Pending Scope
Browser Acceptance	Blocked	Unblock local PostgreSQL, run migration/seeder, and visually verify /admin/providers and /admin/system in English & Arabic.
Canvas Agent Routing UI	Partial	Connect React Flow node inspector to the newly added AiProviderRoutingController tables for per-node model selection.
Account Lifecycle (Phase 2)	Pending	Implement user password reset flow, email verification, account deletion, and active session revocation guards.
Production Hardening	Pending	Load & throughput testing (NFR-01), Celery dead-letter queue recovery, production alert observability, and WCAG accessibility.
Live Staging Execution	Optional	Conduct a single real live workflow run using a paid OpenAI/Gemini key in staging to confirm live token streaming.

5. What's the Next Step (Actionable Roadmap)

- Step 1: Unblock Local Database Volume - Run docker compose down -v to clear the stale credential volume, then docker compose up --build -d so Laravel boots and migrates cleanly.
- Step 2: Execute Browser Acceptance - Log in with test@example.com / NasaqDemo2026, verify the admin provider control page in English and Arabic RTL, and run provider probe tests.
- Step 3: Wire Canvas Nodes to Multi-Agent Routing - Expose model selection dropdowns inside individual canvas agent nodes to utilize the August 25 routing tables.
- Step 4: Establish Git Release Baseline - Stage the clean workspace (excluding binary installers and huge zips) and commit Version 1.0 baseline.

6. Project & Distribution File Structure

Below is the complete architectural layout of both the shared distribution package (shr/) and the root project repository:

```
Nasaq AI Project Directory Layout
|-- shr/                                <-- Isolated Distribution Package (0.44 MB)
|   |-- .env                            <-- Pre-configured Demo Mode (Sanitized, Safe)
|   |-- .env.example                    <-- Environment variable template
|   |-- docker-compose.yml              <-- Primary Docker topology (6 microservices)
|   |-- docker-compose.prod.yml         <-- Production Docker topology
|   |-- start.bat                       <-- Windows 1-click startup script
|   |-- start.sh                        <-- Linux/macOS 1-click startup script
|   |-- README.md                       <-- Bilingual (Arabic/English) Quick Start Guide
|   |-- sample_workflow.json            <-- Pre-built sample workflow for quick import
|   |-- frontend/                       <-- React 18 / Vite Frontend Application
|       |-- src/                        <-- Components, Canvas Nodes, Pages, Locales (ar/en)
|       |-- Dockerfile, nginx.conf      <-- Nginx container build configuration
|       |-- package.json, vite.config.js <-- Build & dependency specifications
|   |-- backend-laravel/                <-- Laravel 11 Backend API & Persistence
|       |-- app/                        <-- Controllers, Models, Policies, Provider Probes
|       |-- database/migrations/         <-- Tables: users, workflows, runs, provider_configs
|       |-- routes/api.php               <-- REST API routes, Sanctum auth, internal callbacks
|       |-- Dockerfile, docker/          <-- PHP 8.5 Alpine container & entrypoint scripts
|   |-- ai-service-python/              <-- FastAPI & Celery AI Execution Engine
|       |-- app/agents/                  <-- Researcher, Writer, Video (FFmpeg), Export
|       |-- app/providers/               <-- Normalized OpenAI and Gemini adapters
|       |-- app/tasks/executions.py      <-- Celery background workflow execution worker
|       |-- Dockerfile, requirements.lock <-- Python container definition
|   |-- infrastructure/                  <-- Nginx reverse proxy & automated test scripts
|-- shr.zip                             <-- Single-File Compressed Archive (Ready to Share)
|-- docs/                               <-- System architecture specifications & API contracts
|-- GradProject.md                       <-- Academic Status Report (M2 Dr. Asrar baseline)
|-- REQUIREMENTS_TRACEABILITY.md         <-- Requirements Traceability & Verification Matrix
-- AUDIT_REPORT.md                       <-- Comprehensive system audit report
```

HANDOVER & DISTRIBUTION SUMMARY

The sharing package (shr/) and its compressed archive (shr.zip) contain everything required to spin up and evaluate the Nasaq AI platform via Docker on any machine with zero configuration. No private API keys or unneeded build caches are included. The platform runs in deterministic demo mode by default.